

# Docker Runbook

## Purpose

This document is the operational runbook for starting the entire FinWallet solution with Docker Compose for the first time, validating service health, inspecting logs, managing MSSQL/Redis volumes, rebuilding individual services, and shutting the stack down safely.

Normal Docker topology:

```
Host
|
| http://localhost:8080
v
FinWallet.Gateway
|
+--> FinWallet.Api
|
+--> FakeBank.Api
+--> FakeFraud.Api
+--> FakeCutoff.Api
+--> FakeCampaign.Api
+--> FakeCommunication.Api

FinWallet.Api
|
+--> MSSQL
+--> Redis
+--> Gateway /providers/*
```

Only the Gateway is published to the host in the normal stack. MSSQL, Redis, FinWallet.Api and fake-provider ports stay inside Docker networks. `compose.debug.yml` publishes them only on 127.0.0.1 for local inspection/debugging.

## Docker files

| File                                  | Purpose                                                                                        |
|---------------------------------------|------------------------------------------------------------------------------------------------|
| <code>compose.yml</code>              | Main Compose definition for the application and infrastructure services.                       |
| <code>compose.debug.yml</code>        | Debug overlay exposing MSSQL, Redis, FinWallet.Api and provider ports on localhost only.       |
| <code>compose.production.yml</code>   | Production-like hardening overrides such as disabling Swagger and stronger restart policies.   |
| <code>docker/Dockerfile.webapi</code> | Shared multi-stage .NET 8 image build for Gateway, FinWallet.Api and all fake Web APIs.        |
| <code>docker/mssql/init-db.sh</code>  | Waits for MSSQL and applies the 001, 002, 003 schema scripts with schema-version tracking.     |
| <code>.env.example</code>             | Template for local Compose environment variables; contains no real production secrets.         |
| <code>.dockerignore</code>            | Removes <code>bin/obj/log/test/docs/local-secret</code> content from the Docker build context. |
| <code>.gitignore</code>               | Prevents build output, logs, secrets and local Docker state from entering Git.                 |

## Docker services

| Service       | Responsibility                                           | Normal host port |
|---------------|----------------------------------------------------------|------------------|
| gateway       | YARP edge gateway, JWT/routing/rate limit/load balancing | 8080             |
| finwallet-api | Main FinWallet Web API                                   | not published    |

| Service            | Responsibility                           | Normal host port |
|--------------------|------------------------------------------|------------------|
| fake-bank          | Bank simulator                           | not published    |
| fake-fraud         | Fraud simulator                          | not published    |
| fake-cutoff        | Cutoff/business-calendar simulator       | not published    |
| fake-campaign      | Campaign simulator                       | not published    |
| fake-communication | SMS/Email simulator                      | not published    |
| mssql              | Financial source of truth                | not published    |
| mssql-init         | One-shot schema initialization container | none             |
| redis              | OTP/transient support state              | not published    |

## Persistent volumes

| Volume                 | Content                       | Survives docker compose down? |
|------------------------|-------------------------------|-------------------------------|
| finwallet_mssql_data   | SQL Server database/log files | Yes                           |
| finwallet_mssql_backup | SQL backup area               | Yes                           |
| finwallet_redis_data   | Redis AOF/RDB persistence     | Yes                           |

Application containers are stateless. Application logs are not persisted to application-specific named volumes; they go to stdout/stderr and Docker log rotation is configured.

### 1. Verify prerequisites

```
docker --version
docker compose version
```

What it does:

- Confirms Docker Engine/Desktop CLI is available.
- Confirms the Compose v2 plugin is installed.
- If these fail, fix Docker before starting FinWallet.

On Windows Docker Desktop, use Linux containers.

### 2. Change to the repository root

```
cd finwallet
```

Run the following commands from the directory containing FinWallet.sln, compose.yml, and .env.example.

### 3. Create the local environment file

PowerShell:

```
Copy-Item .env.example .env
```

Bash:

```
cp .env.example .env
```

What it does:

- Creates a local .env from the version-controlled template.
- .env is ignored by Git and must not be committed.

Replace at least these values with strong local values:

```
MSSQL_SA_PASSWORD
```

```
REDIS_PASSWORD
JWT_SIGNING_KEY
REGISTRATION_OTP_PEPPER
INTERNAL_SERVICE_KEY
DOWNSTREAM_SERVICE_KEY
```

Do not use .env as the production secret-management mechanism. Use orchestrator/secret-store injection in production.

## 4. Validate Compose before starting containers

```
docker compose --env-file .env config --quiet
```

What it does:

- Resolves environment substitutions.
- Validates YAML/Compose syntax.
- Fails early when a required environment variable is missing.
- Does not start containers or modify data.

To print the resolved model:

```
docker compose --env-file .env config
```

Warning: resolved output can contain secrets. Do not redirect it into shared CI logs or files.

List services:

```
docker compose --env-file .env config --services
```

## 5. Build images

```
docker compose --env-file .env build --pull
```

What it does:

- Builds Gateway, FinWallet.Api and fake API images using the multi-stage Dockerfile.
- --pull checks for a newer base image matching the configured tag.
- NuGet restore uses the BuildKit cache.
- Does not start containers.

For a fully clean build:

```
docker compose --env-file .env build --pull --no-cache
```

This is slower and should not be the default development workflow.

## 6. Start the entire system

```
docker compose --env-file .env up -d --build
```

What it does:

- 1 Builds/pulls required images.
- 2 Creates finwallet-data and finwallet-backend networks.
- 3 Creates or reuses MSSQL and Redis named volumes.
- 4 Waits for MSSQL health checks.
- 5 Runs mssql-init, which creates the FinWallet database and unapplied schema versions.
- 6 Starts FinWallet.Api after Redis is healthy and DB initialization completes.
- 7 Starts fake providers and Gateway.
- 8 -d runs the stack in detached mode.

The first build/pull takes longer than subsequent starts.

## 7. Inspect container status

```
docker compose --env-file .env ps
```

Expected state:

- mssql: Up / healthy
- redis: Up / healthy
- mssql-init: Exited (0)
- API and Gateway services: Up

mssql-init being Exited (0) is expected because it is a one-shot migration job.

Docker-level view:

```
docker ps --filter name=finwallet
```

## 8. Verify the Gateway health endpoint

```
curl http://localhost:8080/health/live
```

Windows PowerShell:

```
Invoke-RestMethod http://localhost:8080/health/live
```

If this responds, the host-to-Gateway path is available.

When Swagger is enabled in Docker/Development:

```
http://localhost:8080/swagger
```

Normal client API calls should use Gateway at <http://localhost:8080>.

## 9. Follow logs

Entire stack:

```
docker compose --env-file .env logs -f
```

Gateway:

```
docker compose --env-file .env logs -f gateway
```

FinWallet API:

```
docker compose --env-file .env logs -f finwallet-api
```

MSSQL:

```
docker compose --env-file .env logs -f mssql
```

Redis:

```
docker compose --env-file .env logs -f redis
```

Last 200 lines:

```
docker compose --env-file .env logs --tail 200 gateway finwallet-api
```

-f follows new log entries. Ctrl+C exits log following only; detached containers continue running.

Docker json-file log rotation is configured. Application logs are sent to stdout/stderr rather than durable files inside application containers.

## 10. Verify MSSQL

```
docker compose --env-file .env exec mssql bash -lc '
SQLCMD=$(command -v /opt/mssql-tools18/bin/sqlcmd || command -v /opt/mssql-tools/bin/sqlcmd)
&&
```

```
$SQLCMD -S localhost -U sa -P "$MSSQL_SA_PASSWORD" -C -d FinWallet -Q "SELECT DB_NAME() AS  
DatabaseName; SELECT Version, AppliedAt FROM dbo.SchemaVersions ORDER BY Version;"
```

This:

- connects from inside the SQL Server container;
- confirms the FinWallet database exists;
- lists applied 001, 002, 003 schema versions.

Re-run the migration check manually with:

```
docker compose --env-file .env run --rm mssql-init
```

Already-recorded schema versions are skipped.

## 11. Verify Redis

```
docker compose --env-file .env exec redis sh -lc 'redis-cli -a "$REDIS_PASSWORD"  
--no-auth-warning ping'
```

Expected:

PONG

Persistence details:

```
docker compose --env-file .env exec redis sh -lc 'redis-cli -a "$REDIS_PASSWORD"  
--no-auth-warning INFO persistence'
```

## 12. Publish debug ports

The normal stack intentionally does not publish backend services. For local debugging:

```
docker compose --env-file .env -f compose.yml -f compose.debug.yml up -d --build
```

Loopback ports become:

|                       |      |
|-----------------------|------|
| Gateway               | 8080 |
| FinWallet.Api         | 8081 |
| FakeBank.Api          | 8082 |
| FakeFraud.Api         | 8083 |
| FakeCutoff.Api        | 8084 |
| FakeCampaign.Api      | 8085 |
| FakeCommunication.Api | 8086 |
| MSSQL                 | 1433 |
| Redis                 | 6379 |

The overlay binds debug ports to 127.0.0.1, not the LAN.

Use Gateway for normal integration calls. Direct backend ports are for inspection/debugging and downstream service-key checks remain active.

## 13. Rebuild/restart one service

If FinWallet.Api code changed:

```
docker compose --env-file .env build finwallet-api  
docker compose --env-file .env up -d --no-deps finwallet-api
```

What it does:

- rebuilds only the FinWallet.Api image;
- recreates only that service;
- --no-deps avoids recreating MSSQL/Redis/Gateway.

For Gateway:

```
docker compose --env-file .env build gateway  
docker compose --env-file .env up -d --no-deps gateway
```

Restart without rebuilding:

```
docker compose --env-file .env restart gateway
```

## 14. Open a shell inside a service

```
docker compose --env-file .env exec finwallet-api /bin/sh
```

The runtime image executes the service as the non-root app user. Do not expect root privileges.

Avoid printing the full environment because it contains secrets.

## 15. Monitor resource usage

```
docker stats
```

It shows CPU, memory, network I/O, block I/O and PID counts.

Compose sets CPU, memory and PID limits for application containers. These are local safety baselines, not production capacity numbers.

## 16. Inspect volumes

```
docker volume ls --filter name=finwallet
```

Inspect individually:

```
docker volume inspect finwallet_mssql_data
docker volume inspect finwallet_mssql_backup
docker volume inspect finwallet_redis_data
```

Normal restart/recreate operations keep these volumes.

## 17. Create an SQL backup

Create a backup:

```
docker compose --env-file .env exec mssql bash -lc '
SQLCMD=$(command -v /opt/mssql-tools18/bin/sqlcmd || command -v /opt/mssql-tools/bin/sqlcmd)
&&
$SQLCMD -S localhost -U sa -P "$MSSQL_SA_PASSWORD" -C -Q "BACKUP DATABASE [FinWallet] TO
    DISK = N''''/var/opt/mssql/backup/FinWallet.bak'''' WITH INIT, CHECKSUM"
'
```

The backup remains in the mssql\_backup named volume.

Get the container ID:

```
docker compose ps -q mssql
```

Copy to the host in Bash:

```
docker cp "$(docker compose ps -q mssql):/var/opt/mssql/backup/FinWallet.bak"
./FinWallet.bak
```

Do not commit backups to Git.

## 18. Request a Redis snapshot

```
docker compose --env-file .env exec redis sh -lc 'redis-cli -a "$REDIS_PASSWORD"
--no-auth-warning BGSAVE'
```

This asks Redis to create a background snapshot. Redis is still not the financial source of truth and this is not a substitute for MSSQL backup.

## 19. Stop and start without deleting containers

Stop:

```
docker compose --env-file .env stop
```

Start:

```
docker compose --env-file .env start
```

stop does not remove containers or volumes.

## 20. Shut down while preserving data

```
docker compose --env-file .env down
```

It:

- removes Compose containers;
- removes Compose networks;
- preserves named volumes;
- allows the next up to reuse MSSQL/Redis data.

## 21. Full reset - DESTRUCTIVE

```
docker compose --env-file .env down -v --remove-orphans
```

It removes:

- containers;
- networks;
- mssql\_data, mssql\_backup, redis\_data named volumes.

Result: local database, ledger, customer/auth data, Redis state and SQL backup volume are deleted.

Use this only for a deliberate clean local reset.

## 22. Reset Redis only

```
docker compose --env-file .env down
docker volume rm finwallet_redis_data
docker compose --env-file .env up -d
```

MSSQL data remains intact.

## 23. Reset MSSQL only

```
docker compose --env-file .env down
docker volume rm finwallet_mssql_data finwallet_mssql_backup
docker compose --env-file .env up -d
```

This permanently removes local financial data. Do not use outside disposable local development.

## 24. Inspect and clean Docker disk usage

```
docker system df
```

Build cache cleanup:

```
docker builder prune
```

Unused image cleanup:

```
docker image prune
```

docker system prune -a --volumes is much more destructive and can remove resources used by other projects; it should not be a routine command.

## 25. Use the production-like overlay

```
docker compose \
  --env-file .env \
  -f compose.yml \
  -f compose.production.yml \
  up -d --build
```

The overlay:

- sets ASPNETCORE\_ENVIRONMENT=Production;
- disables Swagger;
- strengthens restart policy;
- enables HSTS configuration.

This is not a replacement for a real production platform. Real production also requires TLS ingress, external secret management, WAF/DDoS controls, image digest pinning, vulnerability scanning, backup/restore procedures, monitoring/alerting and network policy.

## 26. Update the running stack after code changes

```
git pull
docker compose --env-file .env build --pull
docker compose --env-file .env up -d
```

When a new schema script is added, mssql-init must also be updated to recognize the new migration. Merely adding a SQL file is not sufficient.

## 27. Common problems

### Port 8080 already in use

```
docker compose --env-file .env ps
```

Change .env:

```
GATEWAY_PORT=8090
```

Run up -d again. Gateway becomes http://localhost:8090.

### MSSQL unhealthy

```
docker compose --env-file .env logs --tail 300 mssql
```

Check:

- SA password complexity;
- Docker memory availability;
- damaged/incompatible volume;
- restart loop.

### mssql-init failed

```
docker compose --env-file .env logs mssql-init
```

Fix the schema error, then run:

```
docker compose --env-file .env run --rm mssql-init
```

### Redis unhealthy

```
docker compose --env-file .env logs redis
docker compose --env-file .env exec redis sh -lc 'redis-cli -a "$REDIS_PASSWORD"
--no-auth-warning ping'
```

### Gateway returns 502/503

```
docker compose --env-file .env logs --tail 200 gateway finwallet-api fake-bank fake-fraud
fake-cutoff fake-campaign fake-communication
```

Check:

- downstream service is Up;



- Docker service DNS address is correct;
- internal/downstream keys match;
- Gateway health checking has not marked the destination unhealthy.

## Required secret is missing

Compose fails with a message similar to:

```
required variable ... is missing a value
```

Confirm .env exists at repository root and required keys are non-empty.

## 28. Minimum daily development command set

First day:

```
cp .env.example .env
docker compose --env-file .env config --quiet
docker compose --env-file .env up -d --build
docker compose --env-file .env ps
docker compose --env-file .env logs -f gateway finwallet-api
```

At the end of the day, preserve data:

```
docker compose --env-file .env down
```

Next day:

```
docker compose --env-file .env up -d
docker compose --env-file .env ps
```

For a full disposable local reset:

```
docker compose --env-file .env down -v --remove-orphans
docker compose --env-file .env up -d --build
```

Remember: the reset command deletes local MSSQL and Redis data.